ZP-C Illustrated Companion

Why zero is an informational singularity

Information Theory

ZP Companion | Initial: April 2026 | Current: 2.12, November 2026

This companion explains the ideas in plain language with diagrams and real-world examples. It is not the formal document — every claim here restates a result already proved in the corresponding technical document. Consult that document for the authoritative mathematics.

What Is ZP-C Doing?

ZP-C establishes that zero is an informational singularity: in $\mathbb{Q}_2$, the surprisal of states approaching zero grows without bound (Route 2 below). ZP-C also uses a second, separate measure, Kolmogorov complexity (Route 1), which is not an argument for the singularity at zero.

Route 1 — Kolmogorov complexity (algorithmic): How long must a program be to describe a configuration? At the incompressibility threshold P_0, no shorter description exists — the string must be specified in full, every bit. This route is about one string at P_0, not about zero in $\mathbb{Q}_2$.

Route 2 — 2-adic surprisal (probabilistic): How much probability mass does a state carry? As a state approaches zero in $\mathbb{Q}_2$, its probability approaches zero, and its surprisal $I(x) = -\log_2 P(x)$ approaches infinity. Informational cost is unbounded.

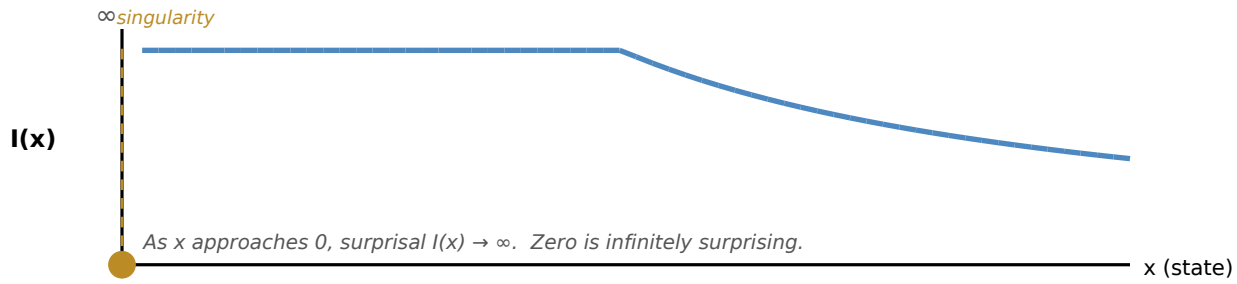
Neither measure knows about the other. K counts program lengths; I(x) counts probability. Depth does not imply incompressibility: the bridge claiming it does is refuted (counterexample $S_n = 2^n$, ZP-I § II.B). ZP-C operates in $\mathbb{Q}_2$ (from ZP-B) throughout — smooth calculus tools are not valid on a totally disconnected space.

Real-world example — ZIP compression hitting a wall

Some files compress a lot; others barely at all. A file that cannot be compressed further has hit its incompressibility threshold — Route 1. Route 2 measures something else: a file holding 2^n in binary, a 1 followed by n zeros, is 2-adically deep (2 divides it n times), yet a short program prints it. Deep is not the same as incompressible.

Surprisal: The Cost of a Transition

The surprisal of state x is $I(x) = -\log_2 P(x)$: how surprising it is to observe x. As x approaches 0, P(x) approaches 0, and I(x) approaches infinity. Zero is infinitely surprising — it cannot be reached by any path of finite informational cost.



Surprisal I(x) as x approaches 0 (amber singularity). Every path toward 0 accumulates unbounded informational cost.

Real-world example — A perfectly random password

To describe a truly random password, you have to send every character — you can't summarize it. That is Route 1: the string is incompressible. Route 2 measures depth instead: zero in $\mathbb{Q}_2$ is the limit of ever-deeper states, with infinite depth and infinite surprisal, and a state can be deep while still easy to summarize.

Where the Two Routes Differ

Kolmogorov complexity K and 2-adic surprisal I(x) are genuinely different tools. K measures how long a program must be to reproduce a string — it is a property of algorithms. I(x) measures how rare a state is — it is a property of probability distributions. They share no common definition and were developed in entirely separate branches of mathematics.

Depth does not imply incompressibility. The claim that 2-adic depth growing without bound carries K to its threshold is refuted (counterexample $S_n = 2^n$, ZP-I § II.B); T-IZ never depended on it (it goes through t_iz_cauchy). ZP-C uses K and I as two separate arguments, not as confirmation of one fact.

What the Singularity Shows, and What It Does Not (L-INF)

The surprisal graph shows that I(x) goes to infinity as x approaches 0. ZP-C makes this precise with Lemma L-INF (Unbounded Surprisal of ⊥, Lean l_inf): for every finite bound there is a depth n of the binary ball hierarchy whose surprisal exceeds it. It is about 2-adic depth, not about the incompressibility threshold P_0; the bridge between the two is refuted (ZP-I § II.B).

That is the depth chart: each depth has a finite surprisal, and there is no largest. The point chart is the limit itself: 0 in $\mathbb{Q}_2$, the one point in every ball of the chain centred at 0, where the 2-adic additive valuation is infinite (padic_addVal_bot). The initial machine state c_0, ⊥ of the machine-state type, corresponds to that point. From the two charts together the framework argues that no finite external interpreter can hold ⊥ as a static description. That argument does not show that anything is running. The step from unbounded surprisal to executing is a bridge principle of its own, which the framework does not adopt (ZP-E, DA-1 Path 2). That the configuration reaching P_0 is a running machine's current configuration, and not an inert string, is what the framework's occurrence commitment asserts (ZP-E). Given it, the L-RUN argument below

applies.

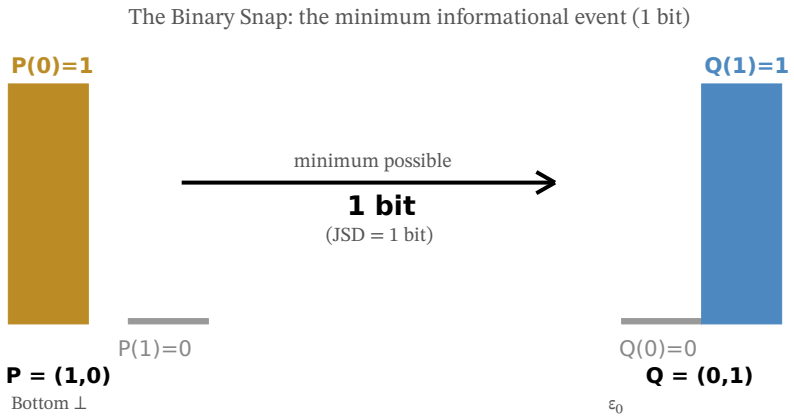
The branching measure — the way ZP-C assigns probabilities to states in D4 — is a representational commitment (RP-2 in ZP-C). It is well-motivated by the binary structure of AX-B1, but it is a choice, not a derivation. The framework labels it explicitly so readers know where a design decision is being made.

Analogy — A file that cannot be described

A truly incompressible file has no pattern — every bit is essential, nothing can be summarized or shortened. At P_0, the configuration is like this file: no shorter description exists. Such a file can still sit on a disk, never opened: having no shorter description does not make it run. That is why ZP-E treats "the configuration at P_0 is running" as a commitment and not as a consequence of incompressibility (DA-1 Path 3).

The Binary Snap Costs Exactly 1 Bit

The Jensen-Shannon Divergence (JSD) between the bottom state ⊥, P = (1,0), and the minimum non-bottom state ε_0, Q = (0,1), is exactly 1 bit. These distributions are derived from AX-B1 - not assumed. The Snap is the minimum informational event possible.



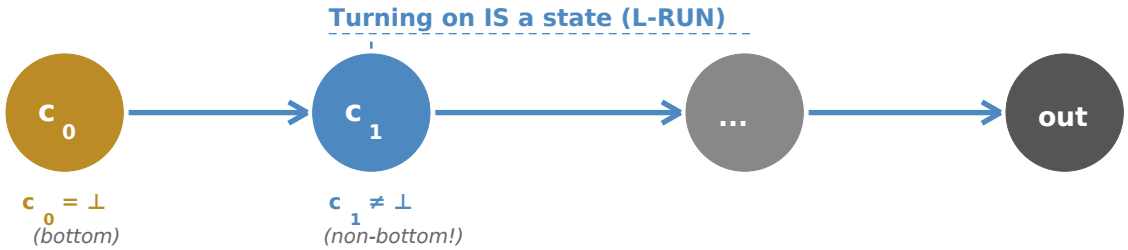
P = (1,0): all probability on non-existence. Q = (0,1): all probability on existence. The informational distance is exactly 1 bit — the minimum possible transition.

Turning On Is a State (L-RUN)

Can a program output ⊥ without passing through any non-bottom intermediate state? The answer is no — because the act of turning on is itself a state.

Any program that executes must pass through a "first instruction fetched" configuration (c_1). That configuration is distinct from the not-yet-running state (c_0). By AX-B1 it is non-bottom. The output being ⊥ does not mean the intermediate states were ⊥.

The identification of c_0 with the bottom state ⊥ is a modeling commitment (CC-2 in ZP-C) — a choice made explicit in the formal document, parallel to CC-1 in ZP-A (which identifies the initial state S_0 with ⊥). It is not derived from the machine definition; it is chosen. Labeling it CC-2 keeps that choice visible.



L-RUN: the transition $c_0 \to c_1$ is a state change off ⊥. Execution itself is a state — independent of output.

Real-world example — Booting a computer

When you press the power button, the machine doesn't jump from "off" to "showing your desktop." It passes through BIOS, POST, bootloader, kernel — each a distinct non-bottom configuration state. Even if programmed to immediately wipe itself and show nothing, it still passed through those intermediate states.

Remember: Computers illustrate the result. L-RUN applies to any Turing machine — the abstract mathematical model of computation. The conclusion is mathematical, not technological.

Two Named Commitments

ZP-C adds explicit labels to two choices the framework makes. Labeling them is not a weakness — it is how the framework keeps track of what is proven versus what is chosen.

CC-2 (Modeling Commitment): The Turing machine initial configuration c_0 is identified with ⊥. This is the same pattern as CC-1 in ZP-A, which identifies the initial state S_0 with ⊥. Neither identification is forced by the definitions — both are deliberate choices that make the multi-layer framework cohere.

RP-2 (Representational Commitment): The branching measure used to assign probabilities to states is a representational choice. It is the natural choice given a binary state space, but it is not the only valid option. Labeling it RP-2 makes the commitment visible to anyone evaluating the framework's assumptions.

Key Result: T-SNAP — The Shape of the Binary Snap Is a Proven Theorem

The derivation chain for the shape: any live computation must execute a first instruction, and that execution is a non-bottom state (L-RUN). No Turing machine program can produce any output without passing through such a non-bottom intermediate state. And any state change off ⊥ is precisely the Binary Snap (ZP-A D2). ZP-E closes the chain as Theorem T-SNAP. The shape of the Binary Snap is no longer assumed — it is derived. That the Snap happens is a separate matter. The framework commits to instantiation occurring: a configuration reaches the incompressibility threshold P_0, as a running machine's current configuration and not as an inert stored description. That is the occurrence commitment; neither incompressibility nor unbounded surprisal derives it. Given it, the machine has left c_0 and is at c_1, by the definition of the first running configuration (D7); DA-1 (ZP-E, closed given DP-2) uses that commitment and does not supply it. Together they give that the Snap happens, which rests on that commitment and is not one of the framework's theorems.